

LIDAR-CAMERA融合

一、功能包介绍 (Project Overview)

1. 核心功能

2. 算法流程 (Pipeline)

如需使用本功能包，交接指南：

fs_fusion_box: FS Driverless 感知融合系统

1. 核心特性 (Key Features)

2. 系统依赖 (Dependencies)

3. 数据接口 (Interfaces)

4. 编译指南 (Build)

5. 配置与参数 (Configuration)

6. 运行指南 (Usage)

7. 可视化调试 (Visualization)

8. 常见故障排除 (Troubleshooting)

debug思路：

可视化延迟问题解决

手动调参全过程记录：

方案1:BBBox

方案2:pointpaint

方案1:

lidar识别出3d的框，包含x，y，长，宽，高；相机识别出2d的框，包含x，y，长，宽，cone类型

通过算法将3d的框投影到2d的画面上，这时对应的位置会有2个框。这两个框的重叠部分的面积与未重叠的的部分的面积的比值计算为置信度。

置信度高：保留，将2d框中的cone类型赋到3d的框上，然后传出x，y，cone类型的话题

置信度低：标上一个unkown，只要表示这里可能有一个锥筒就可，通过下一帧或几帧来辅助确认

如果yolo识别出来的锥筒数小于雷达的，以yolo识别出的为主，对多出来的雷达识别框打上unkown

如果yolo识别出来的锥筒数大于雷达的，则通过画面反推3d的位置，把雷达少检测的锥筒的位置推断出来（精度很低）

一、功能包介绍 (Project Overview)

功能包名称： fs_fusion_box

定位：FS 无人赛车感知系统的核心融合节点 (Perception Fusion Node)

1. 核心功能

本功能包负责接收激光雷达 (LiDAR) 的 **3D 障碍物检测框** 和 相机 (Camera) 的 **2D 识别框**，通过**时空同步与多模态融合算法**，输出带有颜色分类和精确三维坐标的锥筒地图，供下游规划控制模块使用。

2. 算法流程 (Pipeline)

1. 时间同步 (Time Synchronization):
- 使用 `message_filters` 的近似时间同步策略 (ApproximateTime)，确保雷达和相机数据在时间戳上对齐（默认窗口 0.1s）。
2. 空间投影 (Spatial Projection):
- 利用外参矩阵 ($T_{lidar \rightarrow camera}$) 和相机内参 (K)，将雷达 3D 框的 8 个角点投影到相机 2D 图像平面，生成“雷达投影框”。
3. 数据关联 (Data Association):
- 计算“雷达投影框”与“YOLO 2D 框”的 **IoU (交并比)**。

◦ 若 $IoU > \text{阈值}$ ，判定为同一物体，将相机的**颜色属性**赋予雷达的**空间坐标**。
4. 补漏与容错 (Recovery & Fallback):
- **高置信度**：雷达相机均检测到 -> 融合输出（位置准、颜色准）。

◦ **雷达单源**：相机未检出或在视野外 -> 输出 Unknown 颜色锥筒（保位置）。

◦ **相机单源**：雷达漏检（通常在远端） -> 基于单目几何原理（假设地面平坦）反推 3D 坐标（保召回率）。

- 输入 (Sub):
- /perception/lidar/boxes_3d (`vision_msgs/Detection3DArray`)

/perception/camera/boxes_2d (`vision_msgs/Detection2DArray`)
- 输出 (Pub):
- /perception/fusion/map (`drd25_msgs/Map`)：包含位置与颜色的锥筒列表，供规划使用。

如需使用本功能包，交接指南：

一、

▼ Bash

Plain Text

```
1  sudo apt update
2  sudo apt install ros-<distro>-vision-msgs      # 视觉消息格式
3  sudo apt install ros-<distro>-cv-bridge         # OpenCV 桥接
4  sudo apt install ros-<distro>-image-transport   # 图像传输
5  sudo apt install ros-<distro>-tf2-eigen         # Eigen 变换库
6  sudo apt install ros-<distro>-message-filters   # 时间同步库
7  sudo apt install libopencv-dev                 # OpenCV 开发库
```

(注：将 `<distro>` 替换为你使用的 ROS 版本，如 `humble` 或 `foxy`)

自定义依赖:

- 必须拥有 `drd25_msgs` 功能包，并且编译通过。否则本包无法找到头文件。

```
~/ros2_ws/  
├── src/  
│   ├── fs_fusion_box/  
│   └── drd25_msgs/  
└── .
```

fs_fusion_box: FS Driverless 感知融合系统

项目简介

fs_fusion_box 是无人方程式赛车（Formula Student Driverless）感知系统的核心融合节点。

该节点采用**多模态融合架构**，接收来自激光雷达的 **3D 聚类结果** 和来自相机的 **2D YOLO 识别框**，通过时空同步与投影匹配算法，输出带有精确三维坐标与颜色分类的锥筒地图，供下游 SLAM 和规划控制模块使用。

1. 核心特性 (Key Features)

• 双适配器架构 (Dual-Adapter Pattern)

内部集成了数据转换层，无缝对接 `test_cone_segmentation` (3D) 和 `cone_interfaces` (2D) 自定义消息，同时保持核心算法库基于标准 `vision_msgs` 的纯净性。

• 鲁棒的时空同步

使用 `ApproximateTime` 策略自动对齐雷达与相机的时间戳（默认容差 0.1s），确保高速运动下的数据一致性。

• 智能补漏 (Recovery Mechanism)

当雷达漏检（如远距离）但相机检出时，基于单目几何原理反推 3D 坐标，最大化感知召回率。

• 可视化调试系统

内置 Rviz MarkerArray 发布者，实时显示融合后的锥筒位置、颜色及类别，支持“所见即所得”的调试。

2. 系统依赖 (Dependencies)

2.1 自定义消息包 (工作空间内必须包含)

在编译本包之前，请确保你的 `src` 目录下存在并已编译以下功能包：

- `drd25_msgs`: 定义了最终输出的 `Map` 和 `Cone` 消息。
- `test_cone_segmentation`: 提供 3D 雷达聚类输入 (`ThreeDConeArray`)。
- `cone_interfaces`: 提供 2D YOLO 检测输入 (`ConeArray`)。

2.2 系统级依赖

- ROS 2 (Humble 或 Foxy)
- OpenCV (`libopencv-dev`)
- Eigen3
- PCL (Point Cloud Library)

2.3 ROS 常用依赖安装命令

▼ Bash

Plain Text

```
1 sudo apt update
2 sudo apt install ros-$ROS_DISTRO-vision-msgs ros-$ROS_DISTRO-cv-bridge ros
  -$ROS_DISTRO-message-filters ros-$ROS_DISTRO-image-transport ros-$ROS_DISTR
    O-tf2-eigen
```

3. 数据接口 (Interfaces)

3.1 订阅话题 (Subscribers)

内部话题名	重映射目标 (Launch)	消息类型	说明
<code>perception/lidar/cones_custom</code>	<code>/cone_detection_custom</code>	<code>test_cone_segmentation/ThreeDConeArray</code>	3D 激光雷达聚类结果
<code>perception/camera/cones_custom</code>	<code>/yolo/cones</code>	<code>cone_interfaces/ConeArray</code>	2D YOLO 识别结果 (带颜色)

3.2 发布话题 (Publishers)

话题名	消息类型	说明
<code>/perception/fusion/map</code>	<code>drd25_msgs/Map</code>	[核心输出] 融合后的锥筒地图列表
<code>/fusion/markers</code>	<code>visualization_msgs/MarkerArray</code>	[调试输出] Rviz 可视化 (圆柱体)

4. 编译指南 (Build)

1. 进入工作空间
2. 安装依赖 (rosdep)

```
rosdep install --from-paths src --ignore-src -r -y
```

3. 编译

由于存在自定义消息依赖，建议使用以下命令确保编译顺序正确：

```
colcon build --packages-up-to fs_fusion_box
```

（如果没有安装CSF，那么第一次编译的时候会遇到如下报错，即检测不到CSF.h文件

```
In file included from /home/jay/data/driverless26/perception_ws/src/cone_segmentation_test_3d/src/test_cone_segmentation/src/test_cone_segmentation.cpp:1:
/home/jay/data/driverless26/perception_ws/src/cone_segmentation_test_3d/src/test_cone_segmentation/include/test_cone_segmentation/test_cone_segmentation.hpp:31:10: fatal error: CSF.h: 没有那个文件或目录
   31 | #include <CSF.h>
      |
compilation terminated.
gmake[2]: *** [CMakeFiles/cone_segmentation_node.dir/build.make:76: CMakeFiles/cone_segmentation_node.dir/src/test_cone_segmentation.cpp.o] 错误 1
gmake[1]: *** [CMakeFiles/Makefile2:594: CMakeFiles/cone_segmentation_node.dir/all] 错误 2
gmake: *** [Makefile:146: all] 错误 2
---
Failed <<< test_cone_segmentation [6.62s, exited with code 2]
```

那么需要按照如下步骤安装CSF（written by @陶宇科）：

(1) . 下载源码，从 GitHub 克隆 CSF 的官方仓库：

```
▼ Bash C++ |
1 cd ~
2 git clone https://github.com/jianboqi/CSF.git
```

(2) . 编译与安装CSF 使用标准 CMake 构建系统。建议将其安装到系统路径（/usr/local），以便后续多个项目共用。

```
▼ Bash C++ |
1 cd CSF
2
3 # 生成构建配置
4 mkdir build && cd build
5
6 # 编译（使用多核加速）
7 make -j$(nproc)
8
9 # 将头文件和库文件安装到系统目录
10 sudo make install
```

(3) . 配置动态链接库安装完成后，系统需要刷新动态库缓存，否则在运行程序时可能会报错 error while loading shared libraries: libCSF.so。

```
▼ Bash C++ |
1 sudo ldconfig
```

CSF安装编译完成)

除此之外要注意yolo的部分的启动（cone_ws中readme所示），因为写的是绝对路径，所以你需要修改launch文件中的yolo部分启动的代码，把对应的代码改为正确的路径（model_path）

yolo环境配置

0) 环境准备（Humble 使用 Python 3.10）

****重要：**** ROS2 Humble 依赖 Python 3.10，请不要在 conda 的 Python 中运行 ROS2 节点。

```

```bash
conda deactivate

conda deactivate # 需要时多执行一次, 直到 (base) 消失

source /opt/ros/humble/setup.bash

python3 -V # 应显示 3.10.x
```

```

安装运行依赖（只需一次）：

```

```bash
python3 -m pip install ultralytics opencv-python
```

```

```

# =====
# 1. 配置 2D YOLO 节点
# =====
# 对应命令: ros2 run cone_detector yolo_detector ...
# 你的模型路径
model_path = '/home/ori/demo_total/DRd25_ws-main_demoa/src/cone_ws/runs/cone_yolov8n6/weights/best.pt'

yolo_node = Node(
    package='cone_detector',      # 包名
    executable='yolo_detector',   # 可执行文件名
    name='yolo_detector',
    output='screen',
    parameters=[
        'model_path': model_path,
        'image_topic': '/camera1/image_raw', # 你指定的相机话题
        'conf_threshold': 0.3,
        'max_fps': 10.0
    ]
)

```

4. 刷新环境

```
source install/setup.bash
```

5. 配置与参数 (Configuration)

核心配置文件位于：`fs_fusion_box/config/calibration.yaml`

关键参数说明：

- **lidar_frame**: 可视化时的坐标系名称。当前设置为: "hesai_lidar" (必须与 Rviz Fixed Frame 一致)。
- **sync_window**: 雷达与相机时间同步的最大允许误差 (秒)，默认 0.1。
- **lidar_to_camera_matrix**: [非常重要] 4x4 外参矩阵 (LiDAR -> Camera)。如果投影不准，请优先检查此矩阵。
- **camera_matrix & dist_coeffs**: 相机内参及畸变系数 (已根据标定结果更新)。

6. 运行指南 (Usage)

本系统支持两种启动方式。

方式一：一键启动整个感知系统 (推荐)

使用位于 `src` 根目录下的系统级启动脚本，同时拉起 YOLO、雷达分割和融合节点。

```
ros2 launch src/system_run.launch.py
```

方式二：单独启动融合节点

如果你已经在其他终端手动启动了 YOLO 和雷达节点，可以单独运行融合模块：

```
ros2 launch fs_fusion_box fusion_box.launch.py
```

7. 可视化调试 (Visualization)

为了验证融合效果，请使用 Rviz2。

1. 启动 Rviz2:

输入命令 `rviz2`

2. 关键设置 (Global Options):

- **Fixed Frame:** 手动输入 `hesai_lidar`
- (注意：不要选 `map`，也不要选 `velodyne`，必须与 `calibration.yaml` 和雷达驱动发出的一致)。

3. 添加显示插件:

- 点击左下角 `Add` -> 选择 `MarkerArray`。
- **Topic:** 选择 `/fusion/markers`。

4. 图例说明:

-  **蓝色圆柱:** 蓝色锥筒 (Blue)
-  **黄色圆柱:** 黄色锥筒 (Yellow/Big/Small)
-  **红色圆柱:** 红色/橙色锥筒 (Red/Orange)
-  **灰色圆柱:** 未知类别 (Unknown)

8. 常见故障排除 (Troubleshooting)

Q1: Rviz 里一片黑，Global Status 报错 "No tf data"?

A: 检查 Fixed Frame 是否填写为 `hesai_lidar`。可以使用 `ros2 topic echo /fusion/markers --once | grep frame_id` 确认实际发出的 `frame_id` 是什么。

Q2: 节点运行了，但没有任何输出 (ros2 topic hz 为空)?

A: 时间戳同步失败。请检查雷达和相机是否都有数据输出，且时间戳是否接近。

A: 检查话题名。使用 `ros2 topic list` 确认上游节点发出的话题是否真的是 `/cone_detection_custom` 和 `/yolo/cones`。

Q3: 投影位置严重偏差（点在图像里，但 Marker 和点云对不上）？

A: 99% 是外参矩阵 (`lidar_to_camera_matrix`) 错误。请重新标定，或者检查矩阵是否需要转置/求逆。

初步调试：

初版（补全机制不健全）：

 [fs_fusion_box_node.cpp](#)

▼ node测试效果不错的一版

Plain Text

```
1  #include <Eigen/Dense>
2
3  #include <rclcpp/rclcpp.hpp>
4  #include <message_filters/subscriber.h>
5  #include <message_filters/sync_policies/approximate_time.h>
6  #include <message_filters/synchronizer.h>
7  #include <visualization_msgs/msg/marker_array.hpp>
8  #include <opencv2/core/eigen.hpp>
9
10 // --- 消息头文件 ---
11 #include "drd25_msgs/msg/map.hpp"
12 #include "drd25_msgs/msg/cone.hpp"
13 #include "test_cone_segmentation/msg/three_d_cone_array.hpp"
14 #include "cone_interfaces/msg/cone_array.hpp"
15 #include "vision_msgs/msg/detection3_d_array.hpp"
16 #include "vision_msgs/msg/detection2_d_array.hpp"
17
18 #include "fs_fusion_box/fs_fusion_box_math.hpp"
19
20 using namespace std::chrono_literals;
21
22 namespace fs_fusion_box {
23
24 class FusionNode : public rclcpp::Node {
25 public:
26     FusionNode() : Node("fusion_box_node") {
27         // --- 核心参数 ---
28         this->declare_parameter("sync_window", 0.1);
29         this->declare_parameter("lidar_frame", "hesai_lidar");
30
31         // [核心修改 1] 匹配宽容度 (解决近处灰色多的问题)
32         // 设为 25.0 (比较大), 确保近处就算外参歪了也能匹配上颜色
33         this->declare_parameter("match_box_padding", 25.0);
34
35         // [核心修改 2] 删除了所有的 recover_conf_threshold 和 recover_max_di
stance
36         // 回归到第一版: 不做限制, 只要 YOLO 看到就敢补全, 保证最远检测距离。
37
38         // [内参]
39         this->declare_parameter("image_width", 640);
40         this->declare_parameter("image_height", 480);
41         this->declare_parameter("camera_matrix.fx", 500.0);
42         this->declare_parameter("camera_matrix.fy", 500.0);
43         this->declare_parameter("camera_matrix.cx", 320.0);
44         this->declare_parameter("camera_matrix.cy", 240.0);
45         this->declare_parameter("dist_coeffs", std::vector<double>({0,0,
0,0,0,0}));
46
47         // [外参]
48         this->declare_parameter("lidar_to_camera_matrix", std::vector<dou
ble>());
49
50         // --- 初始化 ---
51         marker_pub_ = this->create_publisher<visualization_msgs::msg::Mar
kerArray>("fusion/markers", 10);
52
```

```

        map_pub_ = this->create_publisher<drd25_msgs::msg::Map>("fusion/c
53 ones", 10);
54
55     lidar_sub_.subscribe(this, "perception/lidar/cones_custom");
56     camera_sub_.subscribe(this, "perception/camera/cones_custom");
57
58     typedef message_filters::sync_policies::ApproximateTime<
59         test_cone_segmentation::msg::ThreeDConeArray,
60         cone_interfaces::msg::ConeArray
61     > SyncPolicy;
62
63     sync_ = std::make_shared<message_filters::Synchronizer<SyncPolicy
64 >>(
65         SyncPolicy(10), lidar_sub_, camera_sub_
66     );
67     sync_->registerCallback(
68         std::bind(&FusionNode::callback, this, std::placeholders::_
69 1, std::placeholders::_2)
70     );
71
72     RCLCPP_INFO(this->get_logger(), "Fusion Node Ready (Max Range + A
73 ggressive Matching).");
74     }
75 private:
76     void callback(
77         const test_cone_segmentation::msg::ThreeDConeArray::ConstSharedPt
78 r& custom_lidar_msg,
79         const cone_interfaces::msg::ConeArray::ConstSharedPtr& custom_cam
80 era_msg)
81     {
82         // 1. 3D 数据转换
83         auto standard_lidar_msg = std::make_shared<vision_msgs::msg::Dete
84 ction3DArray>();
85         standard_lidar_msg->header = custom_lidar_msg->header;
86
87         for (const auto& custom_cone : custom_lidar_msg->cones) {
88             vision_msgs::msg::Detection3D det;
89             det.bbox.center.position = custom_cone.center;
90             det.bbox.size = custom_cone.size;
91             standard_lidar_msg->detections.push_back(det);
92         }
93
94         // 2. 2D 数据转换 (保留原始数据用于补全计算)
95         auto standard_camera_msg = std::make_shared<vision_msgs::msg::Det
96 ection2DArray>();
97         standard_camera_msg->header = custom_camera_msg->header;
98
99         for (const auto& box_2d : custom_camera_msg->cones) {
100             vision_msgs::msg::Detection2D det;
101             det.bbox.center.position.x = box_2d.center.x;
102             det.bbox.center.position.y = box_2d.center.y;
103             det.bbox.size_x = box_2d.size.x;
104             det.bbox.size_y = box_2d.size.y;
105
106             vision_msgs::msg::ObjectHypothesisWithPose hyp;
107             hyp.hypothesis.score = box_2d.confidence;

```

```

103         hyp.hypothesis.class_id = std::to_string(box_2d.color);
104         det.results.push_back(hyp);
105         standard_camera_msg->detections.push_back(det);
106     }
107
108     // 3. 构建“匹配专用”的膨胀数据（解决近处灰色多）
109     // 我们把框搞大，只为了让雷达点能落在里面
110     auto matching_camera_msg = std::make_shared<vision_msgs::msg::Detection2DArray>(*standard_camera_msg);
111     double padding = this->get_parameter("match_box_padding").as_double();
112
113     for (auto& det : matching_camera_msg->detections) {
114         det.bbox.size_x += padding;
115         det.bbox.size_y += padding;
116     }
117
118     // 4. 参数加载
119     CalibrationParams params;
120     std::vector<double> matrix_data = this->get_parameter("lidar_to_camera_matrix").as_double_array();
121     if (matrix_data.size() != 16) return;
122     params.T_l2c = Eigen::Map<const Eigen::Matrix<double, 4, 4, Eigen::RowMajor>>(matrix_data.data());
123
124     params.img_w = this->get_parameter("image_width").as_int();
125     params.img_h = this->get_parameter("image_height").as_int();
126     params.K = cv::Mat_<double>(3, 3, CV_64F);
127     params.K.at<double>(0,0) = this->get_parameter("camera_matrix.fx").as_double();
128     params.K.at<double>(1,1) = this->get_parameter("camera_matrix.fy").as_double();
129     params.K.at<double>(0,2) = this->get_parameter("camera_matrix.cx").as_double();
130     params.K.at<double>(1,2) = this->get_parameter("camera_matrix.cy").as_double();
131     std::vector<double> dist = this->get_parameter("dist_coeffs").as_double_array();
132     params.D = cv::Mat(dist).clone();
133
134     // 5. 执行融合（使用膨胀框去匹配）
135     // 这样近处的雷达点就更容易被框住，从而变成彩色
136     auto proj_boxes = project_3d_boxes_to_2d(standard_lidar_msg, params);
137     auto fusion_result = fuse_measurements(proj_boxes, standard_lidar_msg, matching_camera_msg);
138
139     // 6. 执行补全（使用原始框计算位置）
140     // [重要] 这里没有任何 if (score < xx) 的过滤，也没有距离过滤
141     // 这就是为什么第一版检测得远：只要 YOLO 看到，就算没雷达，我们也补出来！
142     auto recovered_cones = recover_missing_cones(fusion_result.unmatched_camera_indices, standard_camera_msg, params);
143
144     // 7. 合并与发布
145     std::vector<drd25_msgs::msg::Cone> final_cones = fusion_result.fused_cones;
146     final_cones.insert(final_cones.end(), recovered_cones.begin(), recovered_cones.end());

```

```

147
148     drd25_msgs::msg::Map map_msg;
149     map_msg.header = custom_lidar_msg->header;
150     map_msg.track = final_cones;
151     map_pub_->publish(map_msg);
152
153     publish_markers(final_cones, custom_lidar_msg->header);
154 }
155
156 void publish_markers(const std::vector<drd25_msgs::msg::Cone>& cone
157 s, const std_msgs::msg::Header& header) {
158     std::string viz_frame = this->get_parameter("lidar_frame").as_str
159 ing();
160     visualization_msgs::msg::MarkerArray markers;
161     visualization_msgs::msg::Marker delete_all;
162     delete_all.action = visualization_msgs::msg::Marker::DELETEALL;
163     delete_all.header.frame_id = viz_frame;
164     markers.markers.push_back(delete_all);
165
166     int id = 0;
167     for(const auto& cone : cones) {
168         visualization_msgs::msg::Marker m;
169         m.header = header;
170         m.header.frame_id = viz_frame;
171         m.ns = "fused_cones";
172         m.id = ++id;
173         m.type = visualization_msgs::msg::Marker::CYLINDER;
174         m.action = visualization_msgs::msg::Marker::ADD;
175         m.pose.position.x = cone.x;
176         m.pose.position.y = cone.y;
177         m.pose.position.z = 0.0;
178         m.scale.x = 0.2; m.scale.y = 0.2; m.scale.z = 0.45;
179         m.color.a = 0.8;
180
181         if(cone.color == 0) { m.color.b = 1.0; }
182         else if(cone.color == 1) { m.color.r = 1.0; }
183         else if(cone.color == 2 || cone.color == 3) { m.color.r = 1.
184 0; m.color.g = 1.0; }
185         else { m.color.r = 0.5; m.color.g = 0.5; m.color.b = 0.5; }
186
187         m.lifetime = rclcpp::Duration::from_seconds(0.2);
188         markers.markers.push_back(m);
189     }
190     marker_pub_->publish(markers);
191 }
192
193 rclcpp::Publisher<visualization_msgs::msg::MarkerArray>::SharedPtr ma
194 rker_pub_;
195 rclcpp::Publisher<drd25_msgs::msg::Map>::SharedPtr map_pub_;
196 message_filters::Subscriber<test_cone_segmentation::msg::ThreeDConeAr
197 ray> lidar_sub_;
198 message_filters::Subscriber<cone_interfaces::msg::ConeArray> camera_s
199 ub_;
200 std::shared_ptr<message_filters::Synchronizer<message_filters::sync_p
201 olicies::ApproximateTime<
202     test_cone_segmentation::msg::ThreeDConeArray, cone_interfaces::ms
203 g::ConeArray>>> sync_;
204 };

```

```
197
198   }
199
200   int main(int argc, char** argv) {
201       rclcpp::init(argc, argv);
202       rclcpp::spin(std::make_shared<fs_fusion_box::FusionNode>());
203       rclcpp::shutdown();
```

▼ 另一个版本（吸铁石版本）

Plain Text

```
1  #include <Eigen/Dense>
2
3  #include <rclcpp/rclcpp.hpp>
4  #include <message_filters/subscriber.h>
5  #include <message_filters/sync_policies/approximate_time.h>
6  #include <message_filters/synchronizer.h>
7  #include <visualization_msgs/msg/marker_array.hpp>
8  #include <opencv2/core/eigen.hpp>
9
10 // --- 消息头文件 ---
11 #include "drd25_msgs/msg/map.hpp"
12 #include "drd25_msgs/msg/cone.hpp"
13 #include "test_cone_segmentation/msg/three_d_cone_array.hpp"
14 #include "cone_interfaces/msg/cone_array.hpp"
15 #include "vision_msgs/msg/detection3_d_array.hpp"
16 #include "vision_msgs/msg/detection2_d_array.hpp"
17
18 #include "fs_fusion_box/fs_fusion_box_math.hpp"
19
20 using namespace std::chrono_literals;
21
22 namespace fs_fusion_box {
23
24 class FusionNode : public rclcpp::Node {
25 public:
26     FusionNode() : Node("fusion_box_node") {
27         // --- 核心参数 ---
28         this->declare_parameter("sync_window", 0.1);
29         this->declare_parameter("lidar_frame", "hesai_lidar");
30
31         // [吸铁石半径] 默认 15.0 像素
32         this->declare_parameter("force_match_radius", 15.0);
33
34         // [新增] 补全门槛
35         // 只有置信度 > 0.7 的框，才有资格进行独立补全
36         this->declare_parameter("recover_conf_threshold", 0.60);
37
38         // [内参]
39         this->declare_parameter("image_width", 640);
40         this->declare_parameter("image_height", 480);
41         this->declare_parameter("camera_matrix.fx", 500.0);
42         this->declare_parameter("camera_matrix.fy", 500.0);
43         this->declare_parameter("camera_matrix.cx", 320.0);
44         this->declare_parameter("camera_matrix.cy", 240.0);
45         this->declare_parameter("dist_coeffs", std::vector<double>({0,0,
0,0,0}));
46
47         // [外参] 不硬编码！必须从 yaml 读取
48         this->declare_parameter("lidar_to_camera_matrix", std::vector<double>());
49
50         // --- 初始化 ---
51         marker_pub_ = this->create_publisher<visualization_msgs::msg::MarkerArray>("fusion/markers", 10);
52         map_pub_ = this->create_publisher<drd25_msgs::msg::Map>("fusion/cones", 10);
```

```

53
54     lidar_sub_.subscribe(this, "perception/lidar/cones_custom");
55     camera_sub_.subscribe(this, "perception/camera/cones_custom");
56
57     typedef message_filters::sync_policies::ApproximateTime<
58         test_cone_segmentation::msg::ThreeDConeArray,
59         cone_interfaces::msg::ConeArray
60     > SyncPolicy;
61
62     sync_ = std::make_shared<message_filters::Synchronizer<SyncPolicy
63 >>(
64         SyncPolicy(10), lidar_sub_, camera_sub_
65     );
66     sync_->registerCallback(
67         std::bind(&FusionNode::callback, this, std::placeholders::_
68 1, std::placeholders::_2)
69     );
70     RCLCPP_INFO(this->get_logger(), "Fusion Node Ready (Anti-Ghost En
71 abled + Magnet + BigOrange).");
72
73 private:
74     void callback(
75         const test_cone_segmentation::msg::ThreeDConeArray::ConstSharedPt
76 r& custom_lidar_msg,
77         const cone_interfaces::msg::ConeArray::ConstSharedPtr& custom_cam
78 era_msg)
79     {
80         // 1. 3D 数据转换
81         auto standard_lidar_msg = std::make_shared<vision_msgs::msg::Dete
82 ction3DArray>();
83         standard_lidar_msg->header = custom_lidar_msg->header;
84         for (const auto& custom_cone : custom_lidar_msg->cones) {
85             vision_msgs::msg::Detection3D det;
86             det.bbox.center.position = custom_cone.center;
87             det.bbox.size = custom_cone.size;
88             standard_lidar_msg->detections.push_back(det);
89         }
90         // 2. 2D 数据转换
91         auto standard_camera_msg = std::make_shared<vision_msgs::msg::Det
92 ection2DArray>();
93         standard_camera_msg->header = custom_camera_msg->header;
94         for (const auto& box_2d : custom_camera_msg->cones) {
95             vision_msgs::msg::Detection2D det;
96             det.bbox.center.position.x = box_2d.center.x;
97             det.bbox.center.position.y = box_2d.center.y;
98             det.bbox.size_x = box_2d.size.x;
99             det.bbox.size_y = box_2d.size.y;
100             vision_msgs::msg::ObjectHypothesisWithPose hyp;
101             hyp.hypothesis.score = box_2d.confidence;
102             hyp.hypothesis.class_id = std::to_string(box_2d.color);
103             det.results.push_back(hyp);
104             standard_camera_msg->detections.push_back(det);
105         }
106     }
107
108
109

```

```

104         // 3. 参数加载 (严格检查)
105         CalibrationParams params;
106         std::vector<double> matrix_data = this->get_parameter("lidar_to_c
amera_matrix").as_double_array();

107
108         // 如果没读到 YAML 里的矩阵, 直接报错并返回 (保护逻辑)
109         if (matrix_data.size() != 16) {
110             RCLCPP_ERROR_THROTTLE(this->get_logger(), *this->get_clock
(), 2000,
111                 "❌ 错误: 未读取到外参矩阵! 请检查 system_run.launch.py 是否正
确加载了 config/fusion_params.yaml");
112             return;
113         }
114         params.T_l2c = Eigen::Map<const Eigen::Matrix<double, 4, 4, Eige
n::RowMajor>>(matrix_data.data());

115
116         params.img_w = this->get_parameter("image_width").as_int();
117         params.img_h = this->get_parameter("image_height").as_int();
118         params.K = cv::Mat::eye(3, 3, CV_64F);
119         params.K.at<double>(0,0) = this->get_parameter("camera_matrix.f
x").as_double();
120         params.K.at<double>(1,1) = this->get_parameter("camera_matrix.f
y").as_double();
121         params.K.at<double>(0,2) = this->get_parameter("camera_matrix.c
x").as_double();
122         params.K.at<double>(1,2) = this->get_parameter("camera_matrix.c
y").as_double();
123         std::vector<double> dist = this->get_parameter("dist_coeffs").as_
double_array();
124         params.D = cv::Mat(dist).clone();
125
126         // 4. 标准融合 (包含吸铁石的前置匹配)
127         auto proj_boxes = project_3d_boxes_to_2d(standard_lidar_msg, para
ms);
128         auto fusion_result = fuse_measurements(proj_boxes, standard_lidar
_msg, standard_camera_msg);

129
130         // =====
131         // [新增] 补全前的“防伪过滤”
132         // -----
133         // 目的: YOLO(0.35) 的低分框可以用来给雷达上色(吸铁石),
134         // 但绝不允许它们在没有雷达的情况下独立生成“鬼影圆柱”。
135         // =====
136         double recover_thresh = this->get_parameter("recover_conf_thresho
ld").as_double();

137
138         auto it = fusion_result.unmatched_camera_indices.begin();
139         while (it != fusion_result.unmatched_camera_indices.end()) {
140             int idx = *it;
141             double score = standard_camera_msg->detections[idx].results
[0].hypothesis.score;

142
143             // 如果置信度 < 0.70, 且没匹配上雷达 -> 认为是误检, 直接剔除!
144             if (score < recover_thresh) {
145                 it = fusion_result.unmatched_camera_indices.erase(it);
146             } else {
147                 ++it;
148             }

```



```
149         }
150
151         // 5. 执行补全 (此时剩下的都是 > 0.7 的高质量框)
152         auto recovered_cones = recover_missing_cones(fusion_result.unmatched_camera_indices, standard_camera_msg, params);
153
154         // =====
155         // 6. 强力吸铁石 (修复近处灰色圆柱)
156         // =====
157         double magnet_radius = this->get_parameter("force_match_radius").as_double();
158         std::vector<drd25_msgs::msg::Cone>& current_cones = fusion_result.fused_cones;
159
160         for (size_t i = 0; i < current_cones.size(); ++i) {
161             if (current_cones[i].color == 4) { // Unknown
162                 // 手动设 Z=0 进行投影
163                 Eigen::Vector4d pt_lidar(current_cones[i].x, current_cones[i].y, 0.0, 1.0);
164                 Eigen::Vector4d pt_cam = params.T_l2c * pt_lidar;
165
166                 if (pt_cam(2) > 0.1) {
167                     double u = (pt_cam(0) * params.K.at<double>(0,0) / pt_cam(2)) + params.K.at<double>(0,2);
168                     double v = (pt_cam(1) * params.K.at<double>(1,1) / pt_cam(2)) + params.K.at<double>(1,2);
169
170                     double min_dist = 1e9;
171                     int best_match_idx = -1;
172
173                     for (size_t j = 0; j < standard_camera_msg->detections.size(); ++j) {
174                         double cx = standard_camera_msg->detections[j].bbox.center.position.x;
175                         double cy = standard_camera_msg->detections[j].bbox.center.position.y;
176                         double dist = std::sqrt(std::pow(u - cx, 2) + std::pow(v - cy, 2));
177
178                         if (dist < min_dist) {
179                             min_dist = dist;
180                             best_match_idx = j;
181                         }
182                     }
183
184                     if (best_match_idx != -1 && min_dist < magnet_radius) {
185                         std::string class_id = standard_camera_msg->detections[best_match_idx].results[0].hypothesis.class_id;
186                         try {
187                             current_cones[i].color = std::stoi(class_id);
188                         } catch (...) {}
189                     }
190                 }
191             }
192         }
193
194         // =====
```

```

195 // 7. 大小橙筒分类 (基于 Z 坐标)
196 // =====
197 int ORANGE_ID = 1;
198 int BIG_ORANGE_ID = 3;
199
200 for (size_t i = 0; i < current_cones.size(); ++i) {
201     if (current_cones[i].color == ORANGE_ID) {
202         if (i < custom_lidar_msg->cones.size()) {
203             double z_coord = custom_lidar_msg->cones[i].center.z;
204             // [修正] 阈值改为 0.35m, 确保能正确识别实物大橙筒
205             if (z_coord > 0.35) {
206                 current_cones[i].color = BIG_ORANGE_ID;
207             }
208         }
209     }
210 }
211
212 // 8. 合并与发布
213 std::vector<drd25_msgs::msg::Cone> final_cones = fusion_result.fused_cones;
214 final_cones.insert(final_cones.end(), recovered_cones.begin(), recovered_cones.end());
215
216 drd25_msgs::msg::Map map_msg;
217 map_msg.header = custom_lidar_msg->header;
218 map_msg.track = final_cones;
219 map_pub->publish(map_msg);
220
221 publish_markers(final_cones, custom_lidar_msg->header);
222 }
223
224 void publish_markers(const std::vector<drd25_msgs::msg::Cone>& cones, const std_msgs::msg::Header& header) {
225     std::string viz_frame = this->get_parameter("lidar_frame").as_string();
226     visualization_msgs::msg::MarkerArray markers;
227     visualization_msgs::msg::Marker delete_all;
228     delete_all.action = visualization_msgs::msg::Marker::DELETEALL;
229     delete_all.header.frame_id = viz_frame;
230     markers.markers.push_back(delete_all);
231
232     int id = 0;
233     for(const auto& cone : cones) {
234         visualization_msgs::msg::Marker m;
235         m.header = header;
236         m.header.frame_id = viz_frame;
237         m.ns = "fused_cones";
238         m.id = ++id;
239         m.type = visualization_msgs::msg::Marker::CYLINDER;
240         m.action = visualization_msgs::msg::Marker::ADD;
241         m.pose.position.x = cone.x;
242         m.pose.position.y = cone.y;
243         m.pose.position.z = 0.0;
244         m.scale.x = 0.2; m.scale.y = 0.2;
245
246         // 可视化: 大橙筒画高一点
247         if (cone.color == 3) {
248             m.scale.z = 0.7;

```

```

249         } else {
250             m.scale.z = 0.3;
251         }
252
253         m.color.a = 0.8;
254         if(cone.color == 0) { m.color.b = 1.0; } // Blue
255         else if(cone.color == 1) { m.color.r = 1.0; } // Red
256         else if(cone.color == 2) { m.color.r = 1.0; m.color.g = 1.0;
    } // Yellow
257         else if(cone.color == 3) { m.color.r = 1.0; m.color.g = 0.5;
    } // Big Orange
258         else { m.color.r = 0.5; m.color.g = 0.5; m.color.b = 0.5; }
    // Unknown
259
260         m.lifetime = rclcpp::Duration::from_seconds(0.2);
261         markers.markers.push_back(m);
262     }
263     marker_pub_>publish(markers);
264 }
265
266     rclcpp::Publisher<visualization_msgs::msg::MarkerArray>::SharedPtr ma
rker_pub_;
267     rclcpp::Publisher<drd25_msgs::msg::Map>::SharedPtr map_pub_;
268     message_filters::Subscriber<test_cone_segmentation::msg::ThreeDConeAr
ray> lidar_sub_;
269     message_filters::Subscriber<cone_interfaces::msg::ConeArray> camera_s
ub_;
270     std::shared_ptr<message_filters::Synchronizer<message_filters::sync_p
olicies::ApproximateTime<
271         test_cone_segmentation::msg::ThreeDConeArray, cone_interfaces::ms
g::ConeArray>>> sync_;
272 };
273
274 }
275
276 int main(int argc, char** argv) {
277     rclcpp::init(argc, argv);
278     rclcpp::spin(std::make_shared<fs_fusion_box::FusionNode>());
279     rclcpp::shutdown();

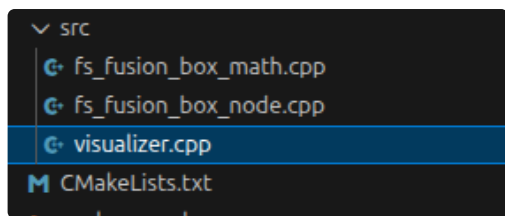
```

[fusion_test.zip](#)

debug思路:

将雷达输入的3d的框转化为2d的框之后，把这个2d的框投影到原来yolo的处理之后的图像上，这样可以通过rviz直观地观察两者匹配的到底准不准

相关代码位于



visualizer.cpp与visualizer.hpp上，与3d点的可视化相独立，可以分别观察

遇到的问题

雷达传入的数据，yolo传入的数据以及yolo处理过之后的画面三者的传入频率相差过大：雷达是10hz左右，yolo传入的数据是8hz左右，而yolo处理过之后的画面是0.5hz左右，这个0.5hz相差太大，导致相关的处理器被“饿死”

解决办法：调高处理器可以忍受的延迟的阈值，最后的成品非常卡顿，但是可以接受，如图所示（因为内参与雷达聚类的方法还没有更新，所以偏差过大是正常现象，仅供参考）：

更进一步

为了解决卡顿的问题，我们决定仅仅打印框而忽视其他图像，这样原来yolo处理后的image的低频率的缺点就可以避免

效果图如下：

[录屏 2026-02-03 15-43-05.mp4](#)

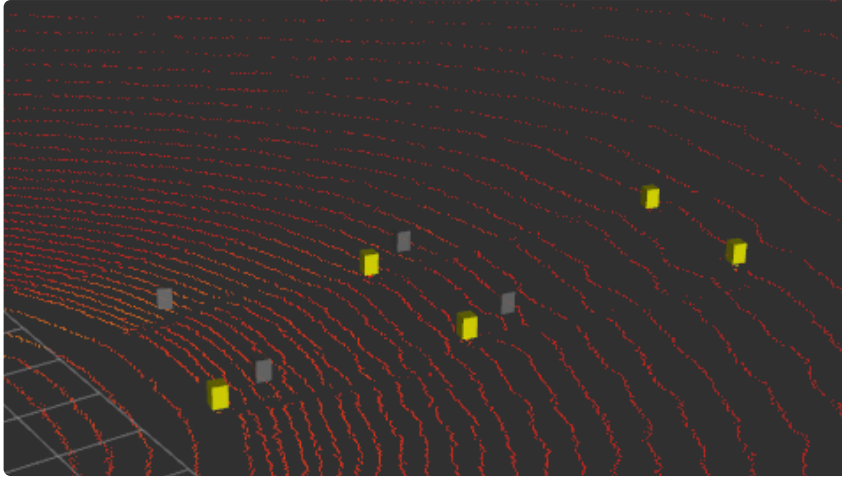
因为万恶的乌班图环境，导致我无法运行最新的雷达的功能包，所以现在的这个包还是老版本的雷达点云检测的包。

[fusion_test.zip](#)

可视化延迟问题解决

现在经过更新了之后，我们发现融合的可视化出了一点问题。即和雷达传来的有一定的位置差。

如下图所示，黄色为准确的位置，灰色为融合输出的位置。



对于这种问题，我思考了几种可能：

通过观察rviz2上的表现，我首先怀疑是延迟的问题。

对于延迟问题，我做出了如下的改进：

(1) 将融合和可视化这两个分为两个单独的进程，同时启动来降低延迟

(2) 原来融合和可视化这两个进程全放在callback里面，复杂计算、文件 I/O、网络请求等阻塞操作会破坏时间确定性，一旦有一个进程卡住，就会阻塞后面所有的进程。所以我修改了代码，让callback仅完成一些数据的处理，以保证实时性。

然而以上的改动确实降低了延迟，但是还是有位置差别

所以第二个想到的是雷达识别时的 w （偏转角），因为雷达的可视化使用了 w ，而我的可视化没有使用 w ，所以我认为问题可能出现在这上面。对此，我让雷达传入的信息包含了他检测到的 w ，然后我的可视化再应用这个 w 来改进他的可视化逻辑。但是最后呈现的效果还是不是很好。

随后我选择了全部按照雷达的可视化逻辑，但是最后还是这样。

我输出了坐标和坐标系，最后得到的都是正确的。所以也不是这个问题。

随后我认为我的匹配机制出现了问题，所以我修改了对于雷达每一帧的匹配机制。

原来的匹配机制：ApproximateTime 近似时间同步，即：

1. 雷达和相机消息分别进入各自的队列
2. 同步器在两个队列里，寻找“时间戳最接近”的一对消息
3. 如果时间差 < 你设置的 sync_window（比如 0.1 秒），就认为匹配成功，触发回调

经过修改之后的匹配机制：LatestTime 机制

不管是雷达来新消息，还是相机来新消息，立刻触发配对。配对规则只有一条：谁最新，就用谁。

其实这并不能解决问题，但是关键在于我多做了一件事：打印出了匹配到的相机和雷达之间时间戳的差。结果我惊讶地发现这两者之间相差了将近750ms。

正是因为这750ms的误差，才导致我们第一种机制存在那么大的延迟：雷达10帧，相机18帧。要补全这750ms的差别，所以两者队列得设定得很长，这也解释了为什么之前队列必须得是10及以上，一旦改小，甚至改到8都几乎配对不上。

我发现是相机的时间戳比雷达更快，所以我在代码里给相机的时间戳加快了750ms，随后发现这两者之间的时间戳相差就只有20，30ms了。

然而，这引申出一个问题：我配对的时候是应该配对时间戳对上的，还是配对时间戳相差750ms左右的呢？

手动调参全过程记录：

首先拉取对应的仓库。因为我们仅需要激光雷达加相机的组合，所以仅拉取这一文件夹。

```
bash
1  # 1. 进入ROS工作空间src
2  mkdir -p ~/calib_ws/src && cd ~/calib_ws/src
3
4  # 2. 初始化稀疏克隆，仅下载目标文件夹
5  git clone --depth 1 --sparse https://github.com/PJLab-ADG/SensorsCalibration.git
6  cd SensorsCalibration
7  git sparse-checkout set lidar2camera
```

随后是配置环境这一块

我仅使用手动标定的那个，即manual_calib文件夹里面的功能

首先是解决CMake 找不到 **Pangolin 0.6** 这个图形界面库的问题

我选择的是下载压缩包，然后在文件内解压的方法

压缩包如下：

[v0.6.zip](#)

放到calib_ws目录下解压

```
▼ bash Bash |
1 # 解压
2 unzip v0.6.zip
3
4 # 重命名文件夹（和之前路径一致）
5 mv Pangolin-0.6 Pangolin
```

然后编译的时候你会发现有报错，根本原因是Pangolin 0.6 在我们的系统的环境下自带的小 bug，解决办法如下：

```
▼ bash Bash |
1 # 打开报错的头文件，加一行代码
2 gedit ~/calib_ws/Pangolin/include/pangolin/gl/colour.h
3
4 #在打开的头文件里面的最上面加上一行代码
5 #include <limits>
```

随后再进入/calib_ws/src/SensorsCalibration/lidar2camera/manual_calib/CMakeLists.txt，修改这个cmake文件成如下格式

```
▼ Bash |
1 cmake_minimum_required(VERSION 2.8.3)
2 project(lidar2camera)
3 set(CMAKE_CXX_FLAGS "-g3 -O -Wall")
4 set(CMAKE_CXX_STANDARD 17)
5
6 find_package(Pangolin 0.6 REQUIRED)
7 include_directories(${Pangolin_INCLUDE_DIRS})
8 link_directories(${Pangolin_LIBRARY_DIRS})
9
10 find_package(PCL REQUIRED)
11 find_package(Boost REQUIRED system)
12 find_package(OpenCV REQUIRED)
13 find_package(PkgConfig REQUIRED)
14 pkg_check_modules(JSONCPP REQUIRED jsoncpp)
15
16 include_directories(${PCL_INCLUDE_DIRS})
17 include_directories(${OpenCV_INCLUDE_DIRS})
18 include_directories(${EIGEN_ROOT})
19 include_directories(${PROJECT_SOURCE_DIR}/include)
20 include_directories(${PROJECT_SOURCE_DIR}/src)
21
22 link_directories(${OpenCV_LIBRARY_DIRS})
23
24 set(CMAKE_ARCHIVE_OUTPUT_DIRECTORY ${PROJECT_SOURCE_DIR}/lib)
25 set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${PROJECT_SOURCE_DIR}/bin)
26
27 add_executable(run_lidar2camera src/run_lidar2camera.cpp)
28 target_link_libraries(run_lidar2camera ${JSONCPP_LIBRARIES} ${PCL_LIBRARIE
S} ${Pangolin_LIBRARIES} ${OpenCV_LIBS} ${Boost_SYSTEM_LIBRARY})
```

随后按下面的过程编译

▼

Bash

```
1  cd Pangolin
2  mkdir build && cd build
3  # 彻底关闭所有Python相关编译
4  cmake .. -DBUILD_PYTHON=OFF -DBUILD_PYTHON_EXAMPLES=OFF
5
6  make -j4
7  sudo make install
8
```

至此环境配置完成

运行时

▼

bash

```
1  # 进入程序所在的bin目录
2  cd ~/calib_ws/src/SensorsCalibration/lidar2camera/manual_calib/bin
3
4  # 运行工具，指定data里的示例数据（顺序不能错！）
5  ./run_lidar2camera \
6  ../data/0.png \
7  ../data/0.pcd \
8  ../data/center_camera-intrinsic.json \
9  ../data/top_center_lidar-to-center_camera-extrinsic.json
```